

Projet : Application Web collaborative de gestion de projets (type plans de communication)

Backend Node.js / Express

Dépendances

- bcrypt : hachage mots de passe
- cookie-parser et jsonwebtoken : gestion des cookies http only cryptés
- cors : gestion des autorisations "Cross-origin resource sharing"
- dotenv : gestion des données d'environnement (lien PostgreSQL, ...)
- multer : gestion des uploads d'images
- pg : gestion de la connexion avec la base de données
- nodemon : seulement côté "dev" pour relancer le serveur automatiquement

Fonctionnement général des routes backend

L'application utilise une architecture modulaire avec séparation des responsabilités :

Structure des routes :

- Toutes les routes sont préfixées par `/api/`
- Chaque module métier possède son propre fichier de routes
- Les routes utilisent le pattern factory avec injection de la connexion PostgreSQL
- Middleware d'authentification pour sécuriser les ressources protégées

Routes disponibles :

- `/api/auth` : Authentification et gestion des utilisateurs
- `/api/header` : Gestion des en-têtes
- `/api/goals` : Gestion des objectifs
- `/api/targets` : Gestion des cibles
- `/api/actions` : Gestion des actions
- `/api/tasks` : Gestion des tâches
- `/api/admin` : Routes d'administration

Sécurisation des ressources statiques :

- Images des goals : `/uploads/goals_images` avec middleware `authenticateGoalImage`
- Images des targets : `/uploads/targets_images` avec middleware `authenticateTargetImage`

- Vérification des droits d'accès avant servir les fichiers statiques

Frontend

Dépendances

- @angular common, compiler, core, forms, platform-browser, platform-browser-dynamic, router : dépendances permettant le fonctionnement de angular
- @tailwindcss/postcss : tailwind pour utiliser les class dans la mise en forme
- gsap : animations javascript avancée
- heroicons : icons ultra optimisés pour l'appli
- rxjs : outils d'angular pour gérer les requêtes et les données
- zone.js : outils d'angular pour gérer la réactivité de l'app
- tslib : bibliothèque type script

Fonctionnement général de l'application client en Single Page App

Composants

- Site Statique de présentation dans le dossier "components/website"
- Authentification et inscription par les composants "components/login" et "components/register"
- Composants du tableau de bord interaction dans "components/dashboard"
- Composants utilisateurs dans "components/layout" (header et navbar)
- Médias stockés dans le dossier "assets" (vidéos, images, etc.)

Fonctionnement

- Services associés aux composants, assurant la communication avec le serveur Express, conçus selon les principes de modularité et de DRY grâce à l'utilisation de modèles partagés avec les composants. Le service http gère les requêtes, le service errors traite la gestion des erreurs, et le fichier environment.ts centralise les constantes utilisables par l'ensemble des services.
- Application configurée grâce au fichier app.config.ts (providers, langue, etc.)
- Routes gérées par le fichier app.routes.ts qui utilise des guards pour sécuriser l'accès aux pages
- Responsive : Adaptation mobile/desktop, mobile-first (classes Tailwind adaptatives, sm:, md:, lg:)
- Utilise Router pour les redirections interne
- Import optimisé : Chaque composant importe uniquement ses dépendances
- Lazy loading : Composants chargés à la demande
- Accessibilité : ARIA labels, rôles sémantiques, navigation clavier, A11y (cdk) pour TrapFocus